

LLM Foundations

Sunday, February 15, 2026 10:29 PM

LLM Architecture

The architecture of LLM models is the **internal structure** and design principles that govern how these models process text data. In simple terms, it's how an LLM is built: what components and layers it contains, how they interact, and how information flows through the model.

From a business perspective, understanding the architecture helps in three ways:

- It clarifies **resource requirements**, such as **computing power** and **storage**.
- It defines how easily an LLM can be **trained, fine-tuned**, or **integrated** into existing **workflows**.
- It highlights limitations and strengths, aiding in better **vendor selection** or build-vs-buy decisions.

Context Pruning: Key Technique to optimize your information more relevant as part of the prompt

Hallucinations
Confused Reasoning
High Token Consumption

Pruning:

Helps the model to understand current intent
Relevant Memory
Active Workflow State

Without Context Pruning Prompt:

Customer Name: Stephen
Address: 219, Ust Street, Tampa
Order History: 5yrs
Invoices: 50 pages
Shipment logs
Current issue: "Where is my shipment"

Context Pruning:

Customer Name: Stephen
Latest Order: 1920-292
Shipment Tracking in Transit
ETA : May 12th
Question: : "Where is my shipment"

Context Pruning:

- 1) Conversation History
- 2) Document (PDF it may 200 pages) - Invoice Approval policy
- 3) Memory - AI Agents maintains memory
 - Prune: expired Memory, duplicated information
 - Keep: current process variables , current business state, user preferences
- 4) Tool Output Pruning - API - return Huge JSON Payload - Prune before sending to LLM

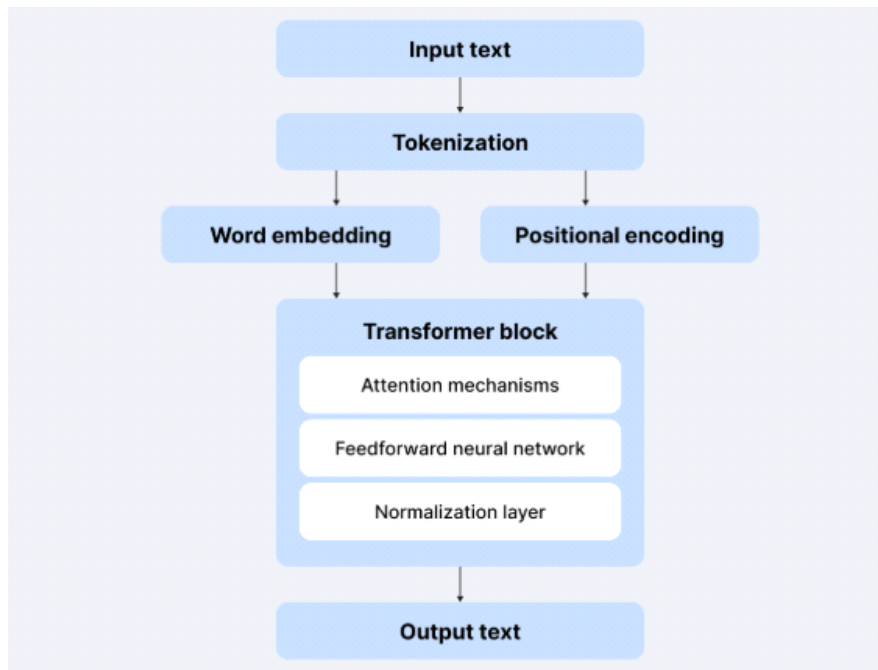
Limits:

Token
CPT - Cost Per Token

Latency/Performance Impact
Reduced accuracy when too much irrelevant context provided

Prompt Compression - Cost Optimization

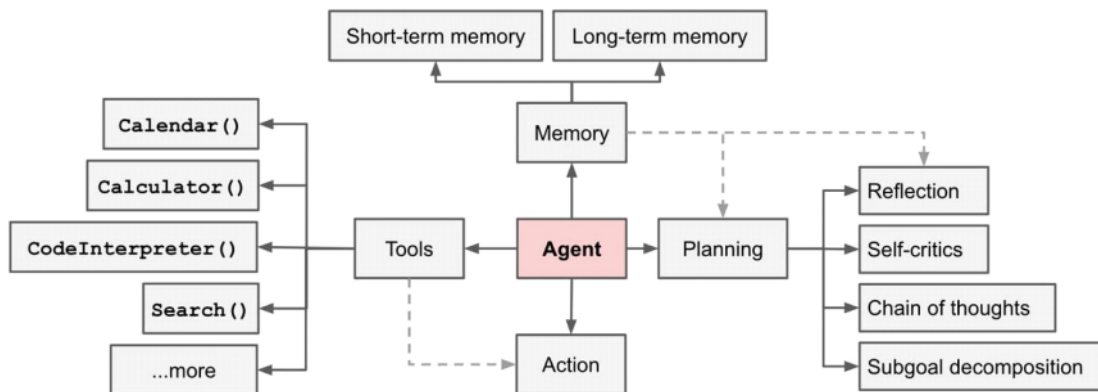
Every large language model is a **combination of architectural components** that work together to **transform raw text into informative output**. These elements are not unique to any one model but are shared across most LLM implementations.



Basic LLM Architecture

Types of LLM Architecture

Type	Main function	Best for	Examples
Encoder-only	Understanding input	Text classification, search, NER, sentiment	BERT
Decoder-only (Causal)	Generating output from input	Text generation, chat, code completion	GPT-3, GPT-4
Encoder-decoder	Transforming input into new output	Translation, summarization, Q&A	T5, BART
Prefix decoder	Task-specific generation with prompt	Instruction following, few-shot learning	FLAN-T5, GPT-3 (prompted)



1) Sequential Agent Pattern

When to Use:

- Data pipelines
- Report generation
- ETL reasoning workflows

2) Supervisor Pattern (Hierarchical)

Supervisor decides:

- Which agent to call
- When to retry
- When to stop

Enterprise Use Case:

- Customer support routing
- DevOps automation
- Incident management AI

3) Event-Driven (Pub/Sub Agents)

Perfect on GCP.

Trigger:

- Pub/Sub event
- Cloud Storage upload
- BigQuery insert
- HTTP request

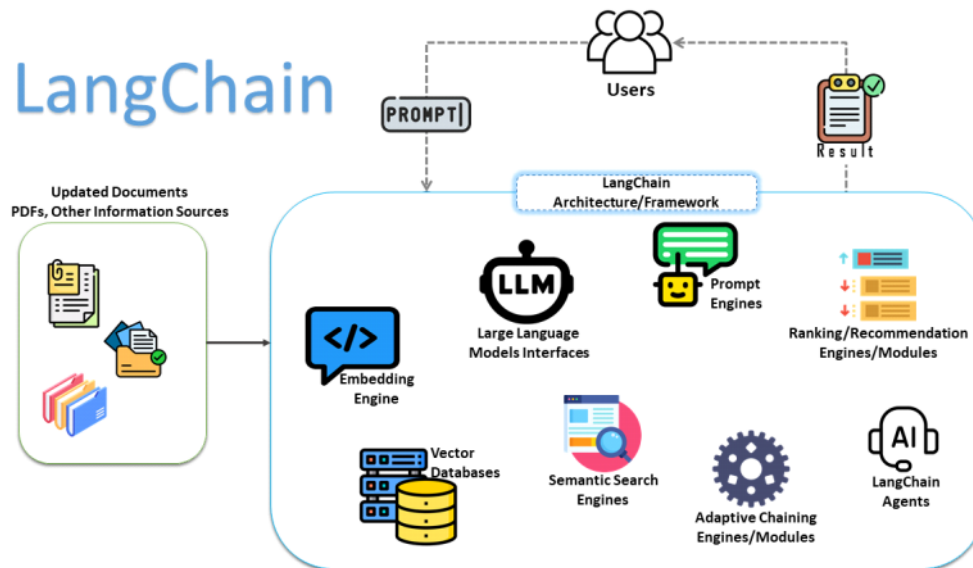
We connect Agents with:

- Cloud Pub/Sub
- Eventarc

LangChain

Why Important?

- Tool abstraction
- Chains
- Agents
- Memory
- Retrieval Augmented Generation (RAG)



MetaGPT

MetaGPT simulates:

- Product Manager
- Architect
- Engineer
- QA

It builds full software projects autonomously.

Enterprise Insight:

MetaGPT = Blueprint for autonomous software factory.

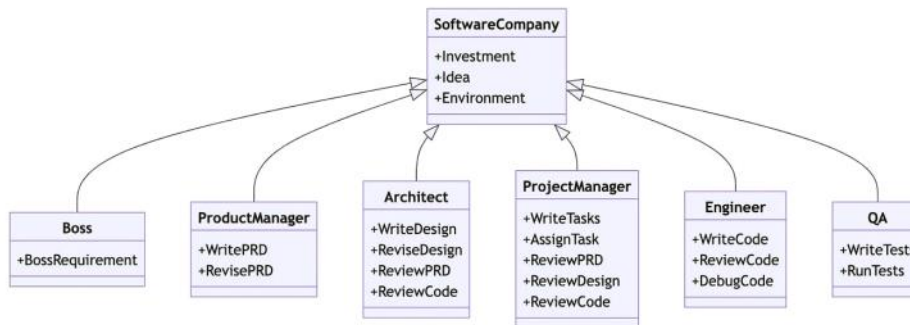
MetaGPT: The Multi-Agent Framework



Assign different roles to GPTs to form a collaborative software entity for complex tasks.

文档 中文版 document English ドキュメント 日本語 MetaGPT License MIT ROADMAP 路线图 WeChat 微信 Follow @MetaGPT

1. MetaGPT takes a **one line requirement** as input and outputs **user stories / competitive analysis / requirements / data structures / APIs / documents**, etc.
2. Internally, MetaGPT includes **product managers / architects / project managers / engineers**. It provides the entire process of a **software company** along with carefully orchestrated SOPs.
 - i. Code = SOP(Team) is the core philosophy. We materialize SOP and apply it to teams composed of LLMs.



Software Company Multi-Role Schematic (Gradually Implementing)

Deployment on GCP

Now we move from Local → Enterprise Deployment

1) Vertex AI Engine

Managed agent runtime.

Why Use It?

- Fully managed
- Observability built-in
- Scalable
- Integrated with Gemini

Deployment Steps

Package agent
Define runtime config
Deploy via gcloud

2) CloudRun

3) Google Kubernetes Engine